

Apache Kafka

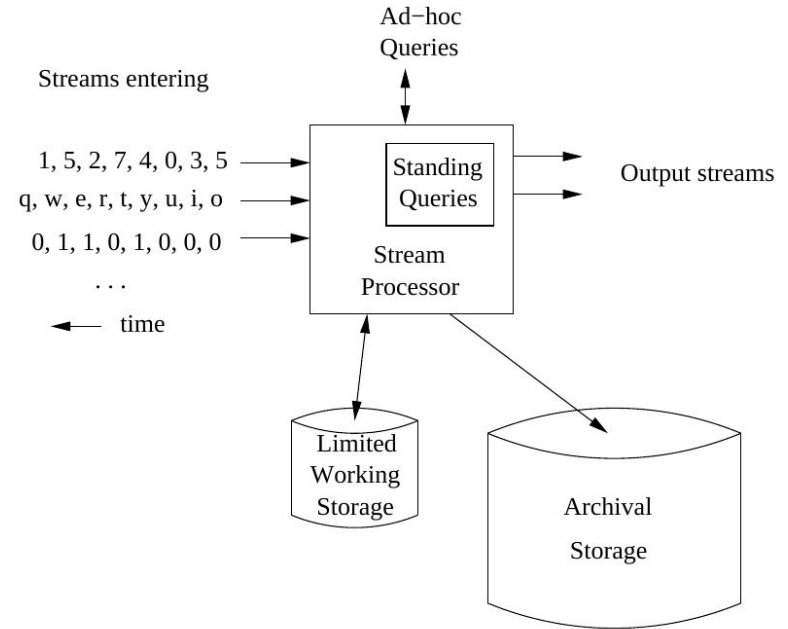
Alka Bhushan

Server Access Link

- <https://lab.samsai.io/student-ui/login>
 - Use email id for login

Real Time Event Processing

- Data arrives in streams
- If not processed or stored will be lost forever
- Not feasible to store all the data
- Data arrival rate is not constant and not under the control of the system
- Not possible to answer queries from archival storage due to time consuming retrieval processes and unavailability of all the data
- Examples: sensors, image, internet and web traffic



Data stream management system [1]

Stream Queries

- Standing queries
 - Max temperature seen so far, avg of last 25 values, alert if temperature exceeds 45 degrees centigrade
- Ad-hoc queries
 - Asked rarely from archival data
 - Sliding window or summary of data
- Processing in real time
 - Sampling
 - filtering

Apache Kafka

- Apache Kafka is a scalable, fault-tolerant, and highly available distributed streaming platform that can be used to store and process data streams.
- Developed by LinkedIn, now an Apache Project
- kafka.apache.org
- Can run on a single node or on a cluster
- Data Streams:
 - sequences of messages/events continuously produced by applications and sequentially and incrementally consumed by other applications.
- supports traditional publish-subscribe messaging, streaming ETL, and data stream processing.
- Data Streaming System:
 - Captures data in real time from event sources like databases, sensors and cloud services etc.
 - Stores for later retrieval
 - Enables real-time and historical stream processing.
 - Routes to different destination technologies
 - Handles varying data arrival rate

Kafka in MLOps

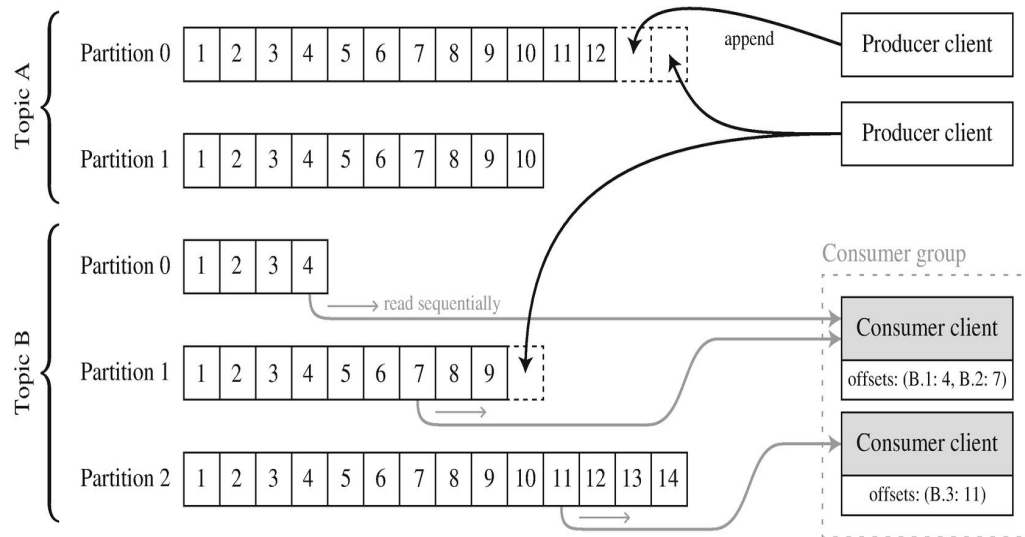
- ML systems continuously generate and consume data from:
 - Applications, User interactions, Sensors and IoT devices, Logs and monitoring systems, Databases and cloud services
- Kafka enables
 - Real time data ingestion
 - Streaming feature engineering
 - Generate rolling average, aggregates, real time transformations
 - Online model inference
 - Consumes events from kafka topics
 - Perform low latency predictions
 - Monitoring and Drift detection
 - Event replay and reprocessing
 - For model retraining , debugging, backfilling

Main Components

- The Kafka Cluster
 - Stores and manages streams
- The connect framework (connect API)
 - ingest data into Kafka and export data streams to external systems like distributed file systems, databases, and others.
- Streams API
 - allows developers to specify sophisticated stream processing pipelines that read input streams from the Kafka cluster and write results back to Kafka.

Main Concepts

- **Event:** a record or a message
 - read/write on kafka is an event
 - Contains key, value, timestamps and optional metadata headers
 - Key and values are of varying length
- **Producers clients (publishers):** publish (write) events to kafka
- **Consumers clients (subscribers):** subscribe to (read and process) these events
- **Brokers:** kafka servers to save messages/events
 - Limit based on time or space be added
- **Topics:** events are organized and stored in topics
 - an append-only sequence of messages, also called a log
- **Partitions :** each topic has multiple partitions
 - Producer specifies partition for the message
 - Automatic, key based etc.
 - Can use any partitioned strategy
 - Default : hash-partitioned by key
- **Offset:** position of message in partition



Main Concepts (contd.)

- Each message is uniquely identified by its partition and offset
- Kafka guarantees strict message ordering within a single partition
 - guarantees that all consumers reading a partition receive all messages in the exact same order as they were appended to the partition.
- No ordering guarantee between messages in different partitions or different topics.
- Topics can be written by multiple producers at the same time
 - Messages written by multiple producers to the same partition are interleaved
- Consumers reading from the same topic form a consumer group
 - Within a group, each consumer can read data from a subset of partitions
 - Each partition is assigned to exactly one consumer from a group
- Different consumer groups are independent of each other

Kafka Brokers

- Messages can be stored for a longer duration
- Kafka supports compacted topics
 - New messages replaces older messages with the same key
 - Compaction is not immediate, triggered in regular intervals
 - Examples: latest feature values per user, latest profile state

Topic	Compacted Topic
Partition 0 Offset 0 : key=user1 → age=20 Offset 1 : key=user2 → age=30 Offset 2 : key=user1 → age=21 Offset 3 : key=user1 → age=22 Offset 4 : key=user2 → age=31	Partition 0 Offset 3 : key=user1 → age=22 Offset 4 : key=user2 → age=31

Kafka Brokers (Contd.)

- **Scaling and Load Balancing**

- Kafka uses distributed partition leadership rather than a single centralized master server.
- A single broker only limits the capacity of a single partition
- A topic can create an arbitrary number of partitions
- Partitions can be reassigned from one broker to another broker to balance the load

- **Fault Tolerance and High Availability**

- Partitions are replicated on multiple brokers
- Each partition has a single leader and multiple followers
- All read and write requests are handled by the leader, while the followers replicate all writes to the leader in the background.
- If the broker hosting the leader fails, Kafka initiates a leader election via ZooKeeper, and one of the followers becomes the new leader. All clients will be updated with the new leader information and send all their read/write requests to the new leader.
- If the failed broker recovers, it will rejoin the cluster, and all hosted partitions become followers.

Kafka Brokers (Contd.)

- Message delivery
 - Kafka does not keep track of consumer's reading progress
 - Consumers can commit their offset to brokers
 - Brokers save offset commits in a special topic called as offset topic, which is partitioned and replicated
 - The offset topic is configured with log compaction enabled
 - This allows Kafka to manage large number of consumers at the same time
- Delivery semantics
 - *At-most-once, at-least-once, exactly-once*
 - *Allows transactions - multi-partition writes*
 - i. Includes more than one messages
 - ii. Consumers cannot read until transaction is complete
 - iii. Messages can be written in more than one partition
 - iv. Follows write all or none messages
 - v. Message which are written in incomplete transaction will be marked as aborted

Kafka Brokers (Contd.)

- Reading from Kafka
 - Consumers commits their offset regularly to Kafka for fault tolerance
 - Follows first commit and processing afterwards or vice versa
 - Transactional consumers
 - Can read all messages and deliver only non aborted messages

Kafka Connect / Connect API

- A framework for integrating Kafka with external system
 - External systems: Distributed file system, databases, key value stores
- Uses connector to communicate with external system
 - Source connector : read from external system and write to Kafka
 - Sink connector: read from Kafka and send to external system
 - Connectors can be created by developers
- Kafka connect can be used as standalone or deployed to a cluster
- To connect with external system, it creates configuration at one of the connect worker
- Allows simple transformations on a single message
 - Custom transformation are possible
 - Streams API is used for complex transformations

Kafka Streams / Stream API

- Is used to build real time stream processing applications
- Provides high level DSLs (Domain Specific Language)
- Supports stateless (filter) and stateful (join, aggregate, window) operators
- Supports exactly once and event time semantics
 - Exactly once: (i) actual processing (ii) writing output (iii) recording progress
 - All three tasks must be performed otherwise 'none'
 - Event time semantics : ingestion time, event time, processing time
- Handles out of order and late arriving data
- Supports streams and tables both
- Supports state management by saving states in a topic which is linear in size by using changelog topics
- fault tolerant and scalable

References

1. <https://kafka.apache.org/43/getting-started/introduction/>
2. Original paper:
https://drive.google.com/file/d/1RGPcKBAAd2BobY6-pVa_QvCTx5YZw8SEZ/view?usp=sharing